

Analisi statica di *xdns.c*

Win32.Mydoom.a — sottosistema DNS per la risoluzione dei server MX

Questo documento raccoglie l'analisi del file **xdns.c** del sorgente Win32.Mydoom.a. L'obiettivo è documentare, attraverso il codice, il ruolo del modulo nel funzionamento complessivo del malware, senza confondere la risoluzione DNS con l'invio SMTP vero e proprio.

1. Funzione del modulo

xdns.c non invia direttamente le email. Il suo compito è ottenere i record **MX (Mail Exchange)** associati al dominio di un destinatario. Questi dati vengono poi utilizzati dal componente SMTP (*xsmtp.c*) per individuare i server a cui tentare la consegna del messaggio.

```
scan.c
↓
raccolta indirizzi email
↓
massmail.c
↓
coda destinatari
↓
msg.c
↓
generazione email + allegato
↓
xdns.c
↓
ricerca record MX
↓
xsmtp.c
↓
connessione SMTP
↓
invio
```

2. Query DNS: tipo MX e classe Internet

```
#define TYPE_MX 15
#define CLASS_IN 1
```

Il codice identifica esplicitamente il tipo DNS **MX (15)** e la classe **IN (1)**. La funzione **mx_make_query()** costruisce manualmente una richiesta DNS, inserendo il nome del dominio nel formato QNAME e specificando il tipo MX.

```
tmp = mx_dns2qname(domain, buf+i); i += tmp;
*(WORD *)(buf+i) = htons(TYPE_MX); i += 2;
*(WORD *)(buf+i) = htons(CLASS_IN); i += 2;
```

3. Comunicazione DNS tramite UDP

```
sock = socket(PF_INET, SOCK_DGRAM, IPPROTO_UDP);

tmp = sendto(sock, buf, i, 0,
             (struct sockaddr *)dns_addr,
             sizeof(struct sockaddr_in));
```

Il modulo crea un socket UDP e invia direttamente la richiesta DNS al server configurato. La risposta viene attesa tramite **select()**, con un timeout di 12 secondi. Sono previsti due tentativi, il primo con la richiesta di ricorsione e il secondo senza.

```
if (loc_retry == 0)
    query_fl = htons(0x0100);
else
    query_fl = htons(0);
```

4. Parsing della risposta DNS

Il malware non si limita a ricevere una risposta: contiene un parser DNS dedicato. **mx_parse_rr()** percorre i resource record e costruisce una lista interna. **mx_decode_domain()** interpreta i nomi DNS, compresi i puntatori utilizzati per la compressione dei nomi.

```
rrcode = ntohs(reply_hdr->flags) & 0x0F;

if (rrcode == 3) {
    *err_stat = 2;
    break;
}

if (rrcode != 0)
    continue;
```

La risposta viene inoltre sottoposta a controllo del codice di risposta DNS. Un dominio inesistente (RCODE 3) viene trattato come errore permanente; altri errori possono portare a nuovi tentativi.

5. Estrazione dei server MX

```
if ((rr1->rr_class != CLASS_IN) ||
    (rr1->rr_type != TYPE_MX) ||
    (rr1->rdlen < 3))
    continue;

mxlist_new->pref =
    ntohs(*(WORD *)(buf+rr1->rdata_offs+0));

mx_decode_domain(
    buf,
    rr1->rdata_offs+2,
    reply_len,
    mxlist_new->mx
);
```

Per ogni record valido il codice estrae due informazioni: la **preferenza** del record MX e il nome del **mail exchanger**. I risultati vengono organizzati in una lista collegata di strutture **mxlist_t**.

Elemento	Ruolo nel codice
rr_type = MX	Seleziona i record che indicano i server di posta
rr_class = IN	Limita la selezione alla classe Internet
pref	Memorizza la preferenza/priorità del record MX
mx	Memorizza il nome del server mail exchanger

6. Primo metodo: API DNS di Windows

```
statusDns = pDnsQuery_A(
    domain,
    DNS_TYPE_MX,
    DNS_QUERY_STANDARD,
    NULL,
    &pQueryResults,
    NULL
);
```

La funzione **getmx_dnsapi()** carica dinamicamente **dnsapi.dll** e recupera **DnsQuery_A** tramite **GetProcAddress()**. Viene quindi eseguita una query standard per record MX. Se la query ha successo, i record vengono convertiti nella struttura interna **mxlist_t**.

7. Secondo metodo: DNS diretto come fallback

```
if ((p = getmx_dnsapi(domain)) != NULL)
    return p;
else
    return getmx_mydns(domain);
```

Se l'API DNS di Windows non produce una lista MX, il codice utilizza un secondo percorso. **getmx_mydns()** usa **GetNetworkParams()** per recuperare i server DNS configurati sul sistema e li interroga direttamente.

```
addr.sin_family = AF_INET;
addr.sin_port = htons(53);
...
mxlist = my_get_mx_list(&addr, domain);
```

La porta **53** è quindi utilizzata per le query DNS. Il codice può provare più server presenti nella lista DNS configurata e interrompe la ricerca quando ottiene una lista MX.

8. Funzione di coordinamento

```
struct mxlist_t *get_mx_list(const char *domain)
{
    struct mxlist_t *p;

    if ((p = getmx_dnsapi(domain)) != NULL)
        return p;
    else
        return getmx_mydns(domain);
}
```

Questa funzione costituisce l'interfaccia principale del modulo: prima tenta il metodo standard tramite l'API Windows e, in caso di fallimento, utilizza il resolver DNS implementato nel sorgente.

9. Gestione della memoria

```
void free_mx_list(struct mxlist_t *p)
{
    struct mxlist_t *q;
    while (p != NULL) {
        q = p->next;
        mx_free(p);
        p = q;
    }
}
```

Le liste dinamiche contenenti i server MX vengono liberate con **free_mx_list()**. Anche la lista intermedia dei resource record DNS viene liberata con **mx_free_rrlist()**.

10. Evidenze comportamentali

Evidenza nel sorgente	Comportamento dimostrato
DNS_TYPE_MX / TYPE_MX 15	Ricerca dei server di posta del dominio
socket(... SOCK_DGRAM ...)	Uso di UDP per il traffico DNS
sendto()	Invio diretto delle query DNS
htons(53)	Uso della porta DNS standard
DnsQuery_A()	Risoluzione tramite API DNS di Windows
GetNetworkParams()	Recupero dei DNS configurati sul sistema
mx_decode_domain()	Parsing dei nomi DNS, inclusa la compressione
pref + mx	Conservazione di priorità e hostname dei server MX
getmx_dnsapi() → getmx_mydns()	Presenza di un metodo DNS alternativo

11. Collegamento con il mass-mailing

Nel comportamento complessivo ricostruito dai sorgenti, **xdns.c** rappresenta il passaggio che collega la coda dei destinatari e la generazione del messaggio alla successiva comunicazione SMTP. Il modulo determina cioè quali server di posta associare al dominio del destinatario.

```
destinatario
↓
dominio
↓
get_mx_list(domain)
↓
record MX
↓
server mail exchanger
↓
xsmtp.c
↓
SMTP
```

Il codice di *xdns.c*, preso isolatamente, dimostra la capacità di risolvere i server MX; la conclusione relativa all'invio di posta deriva dal suo collegamento con *massmail.c*, *msg.c* e *xsmtp.c* analizzati separatamente.

12. Osservazioni sul codice

Il sorgente presenta caratteristiche di codice legacy e alcune anomalie di implementazione. Per il rapporto principale non è necessario basare la valutazione di pericolosità su questi difetti: le evidenze funzionali sopra riportate sono sufficienti a documentare il ruolo del modulo.

Conclusione

xdns.c implementa un sottosistema DNS specifico per ottenere i record MX dei domini dei destinatari. Il malware dispone sia di un metodo tramite le API DNS di Windows sia di un resolver DNS alternativo che interroga direttamente i server DNS configurati. I risultati vengono trasformati in una lista di mail exchanger con relativa preferenza e sono destinati a essere utilizzati dal componente SMTP. Nel contesto dell'insieme dei sorgenti analizzati, questo modulo costituisce quindi un elemento tecnico fondamentale della capacità di mass-mailing di Win32.Mydoom.a.